# Adversarial Face Recognition Attack Project Progress Report

**Name:** Aleeha Ibrar
**Internship:** NASTP Internship – Centaic
**Domain:** Computer Vision & Adversarial Machine Learning
**Report Date:** 21 July 2026

## 1. Current Progress

### Completed

- Studied the fundamentals of adversarial attacks on deep learning models.
- Explored adversarial attack techniques for face recognition systems.
- Reviewed gradient-based attack methods including Fast Gradient Sign Method (FGSM).
- Researched publicly available face recognition datasets.
- Selected the Labeled Faces in the Wild (LFW) dataset for experimentation.
- Set up the Python development environment with PyTorch and required dependencies.
- Explored pretrained deep learning models for feature extraction.
- Implemented a ResNet50-based face feature extractor.
- Implemented the FGSM adversarial attack framework.
- Developed utilities for loading images, generating adversarial examples, and saving attack results.
- Successfully generated adversarial face images for evaluation.
- Implemented a feature-similarity-based evaluation pipeline (cosine similarity and L2 distance) to quantify attack success.
- Implemented a full-dataset evaluation script that runs the epsilon sweep across the entire LFW test split and compiles summary statistics (mean/median/std similarity, success rate with 95% confidence interval) into a CSV report.

### In Progress

- Executing the full-dataset evaluation script on the LFW test split (pending compute/run time).
- Populating the summary statistics table below with the completed run's output.
- Generating result visualizations (ASR vs. epsilon, similarity distributions) from the CSV output.

## 2. Research Sources Explored

### Research Papers

**Explaining and Harnessing Adversarial Examples**
https://arxiv.org/abs/1412.6572

**Intriguing Properties of Neural Networks**
https://arxiv.org/abs/1312.6199

**Towards Deep Learning Models Resistant to Adversarial Attacks**
https://arxiv.org/abs/1706.06083

**Survey on Adversarial Machine Learning**
https://arxiv.org/abs/1810.00069

**FaceNet: A Unified Embedding for Face Recognition and Clustering**
https://arxiv.org/abs/1503.03832

### Official Dataset Sources

**Labeled Faces in the Wild (LFW)**
Type:

- Face Recognition
- Identity Verification

Source:
https://www.kaggle.com/datasets/jessicali9530/lfw-dataset

**VGGFace2 Dataset**
Type:

- Large-scale Face Recognition

Source:
https://www.robots.ox.ac.uk/~vgg/data/vgg_face2/

**CelebFaces Attributes (CelebA)**
Type:

- Face Recognition
- Facial Attribute Prediction

Source:
https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html

## 3. Dataset Selected

### Selected Dataset

### Labeled Faces in the Wild (LFW)

Dataset Source:
https://www.kaggle.com/datasets/jessicali9530/lfw-dataset

### Reason for Selection

- Widely used benchmark dataset for face recognition research.
- Contains 13,233 face images of 5,749 individuals.
- Easily accessible through Kaggle.
- Suitable for evaluating adversarial attacks.
- Compatible with PyTorch and Torchvision.
- Frequently used in academic research.

## 4. GitHub Repositories Explored

### Adversarial Face Recognition Attack Repository

Repository:
https://github.com/mahmoudnafifi/Adversarial-Attacks-on-Face-Recognition
Reason Explored:

- Implements adversarial attacks on face recognition systems.
- Demonstrates FGSM attacks.
- Uses pretrained deep learning models.
- Provides evaluation scripts.

### PyTorch

Repository:
https://github.com/pytorch/pytorch
Reason Explored:

- Deep learning framework.
- GPU acceleration.
- Large research community.
- Extensive documentation.

### Torchvision

Repository:
https://github.com/pytorch/vision
Reason Explored:

- Pretrained computer vision models.
- Image preprocessing utilities.
- Dataset support.
- Easy integration with PyTorch.

### Torchattacks

Repository:
https://github.com/Harry24k/adversarial-attacks-pytorch
Reason Explored:

- Collection of adversarial attack algorithms.
- Includes FGSM, PGD, BIM, CW, DeepFool, and many others.
- Easy PyTorch integration.
- Useful for benchmarking different attacks.

### Adversarial Robustness Toolbox (IBM)

Repository:
https://github.com/Trusted-AI/adversarial-robustness-toolbox
Reason Explored:

- Industry-standard adversarial machine learning toolkit.
- Supports multiple attack and defense methods.
- Compatible with PyTorch and TensorFlow.
- Comprehensive evaluation framework.

## 5. Repository Selected

### Selected Repository

### Adversarial Face Recognition Attack

### Reason

- Implements FGSM adversarial attacks.
- Uses pretrained ResNet50 architecture.
- Well-structured project organization.
- Easy to modify and extend.
- Suitable for evaluating adversarial robustness.
- Supports experimentation with different attack strengths.

## 6. Current Implementation Status

Current workflow:

- Development environment configured.
- Pretrained ResNet50 model integrated.
- Face image preprocessing completed.
- Feature extraction implemented.
- FGSM attack successfully implemented.

- Adversarial face images generated.
- Perturbation visualization completed.
- Image saving and evaluation pipeline developed.
- Feature-similarity evaluation metric implemented.
- Full-dataset evaluation script implemented (execution pending).

## 7. Attack Evaluation Methodology

To address the objective of *evaluating attack success rate using feature similarity metrics*, the following methodology was implemented.

### Evaluation Approach

- For each test image, extract the ResNet50 embedding of the **original** image and the corresponding **adversarial** image.
- Compute the **cosine similarity** and **Euclidean (L2) distance** between the two embeddings.
- Define an attack as **successful** if the cosine similarity drops below a chosen verification threshold (commonly used thresholds in face-verification literature range from 0.5–0.6, tuned per model).
- Aggregate results across the test set to compute an overall **Attack Success Rate (ASR)**.
- Repeat the process for a range of epsilon (perturbation budget) values to study the strength–success trade-off.

### Epsilon Sweep

The FGSM attack is evaluated at multiple perturbation magnitudes to observe how success rate scales with visible distortion:

- $\epsilon = 0.01$ – minimal perturbation.
- $\epsilon = 0.03$ – moderate perturbation.
- $\epsilon = 0.05$ – stronger perturbation.
- $\epsilon = 0.1$ – aggressive perturbation, expected higher success rate with more visible artifacts.

### Per-Batch Evaluation Code

Listing 1: Embedding similarity evaluation for adversarial attack success

```python
import torch
import torch.nn.functional as F

def evaluate_attack_success(model, original_imgs, adversarial_imgs, threshold=0.5):
    """
    Compare embeddings of original vs adversarial images to measure
    adversarial attack success rate using cosine similarity.
    """
    model.eval()
    with torch.no_grad():
        orig_embeddings = model(original_imgs)
        adv_embeddings = model(adversarial_imgs)

    cos_sim = F.cosine_similarity(orig_embeddings, adv_embeddings, dim=1)
    l2_dist = torch.norm(orig_embeddings - adv_embeddings, dim=1)
    success = (cos_sim < threshold).float()
    success_rate = success.mean().item()

    return {
        "cosine_similarity": cos_sim,
        "l2_distance": l2_dist,
        "success_rate": success_rate,
    }
```

## 8. Full-Dataset Evaluation

To address the next objective – *completing a full-dataset evaluation of attack success rate and compiling summary statistics* – the per-batch evaluation above was extended into a full-dataset pipeline (`evaluate_full_dataset.py`) that:

- Iterates over the entire LFW test split in batches, rather than a single sample batch.
- Runs the FGSM attack at each epsilon value in $\{0.01, 0.03, 0.05, 0.1\}$ for every batch.
- Accumulates cosine similarity and L2 distance values per epsilon across all samples.

- Computes, per epsilon: sample count, mean/median/standard deviation of cosine similarity, mean/median/standard deviation of L2 distance, attack success rate, and a 95% confidence interval on the success rate.
- Writes the compiled statistics to a CSV file and prints a console summary table.

**Evaluation Script**

Listing 2: Full-dataset evaluation and summary statistics (evaluate_full_dataset.py)

```
import csv
import numpy as np
import torch
import torch.nn.functional as F
from dataclasses import dataclass, field

EPSILONS = [0.01, 0.03, 0.05, 0.1]
SIMILARITY_THRESHOLD = 0.5

@dataclass
class EpsilonStats:
    epsilon: float
    n_samples: int = 0
    cos_sims: list = field(default_factory=list)
    l2_dists: list = field(default_factory=list)
    successes: list = field(default_factory=list)

    def update(self, cos_sim, l2_dist, success):
        self.cos_sims.extend(cos_sim.detach().cpu().tolist())
        self.l2_dists.extend(l2_dist.detach().cpu().tolist())
        self.successes.extend(success.detach().cpu().tolist())
        self.n_samples += cos_sim.shape[0]

    def summary(self):
        cos, l2, succ = np.array(self.cos_sims), np.array(self.
            l2_dists), np.array(self.successes)
        n = max(len(succ), 1)
        p = succ.mean() if len(succ) else 0.0
        se = np.sqrt(p * (1 - p) / n) if n > 0 else 0.0
        ci_low, ci_high = max(0.0, p - 1.96 * se), min(1.0, p +
            1.96 * se)
        return {
            "epsilon": self.epsilon, "n_samples": self.n_samples,
```

```
            "cos_sim_mean": float(cos.mean()), "cos_sim_median": float(np.median(cos)),
            "cos_sim_std": float(cos.std()),
            "l2_dist_mean": float(l2.mean()), "l2_dist_median": float(np.median(l2)),
            "l2_dist_std": float(l2.std()),
            "success_rate": float(p),
            "success_rate_ci95_low": float(ci_low), "success_rate_ci95_high": float(ci_high),
        }

def run_full_evaluation(model, attack_fn, dataloader, epsilons=EPSILONS,
                        threshold=SIMILARITY_THRESHOLD, device="cpu"):
    all_stats = {eps: EpsilonStats(epsilon=eps) for eps in epsilons}
    for images, _labels in dataloader:
        images = images.to(device)
        for eps in epsilons:
            adv_images = attack_fn(model, images, eps)
            with torch.no_grad():
                orig_emb, adv_emb = model(images), model(adv_images)
            cos_sim = F.cosine_similarity(orig_emb, adv_emb, dim=1)
            l2_dist = torch.norm(orig_emb - adv_emb, dim=1)
            success = (cos_sim < threshold).float()
            all_stats[eps].update(cos_sim, l2_dist, success)
    return {eps: stats.summary() for eps, stats in all_stats.items()}

def write_results_csv(results, output_path):
    fieldnames = ["epsilon", "n_samples", "cos_sim_mean", "cos_sim_median", "cos_sim_std",
                  "l2_dist_mean", "l2_dist_median", "l2_dist_std", "success_rate",
                  "success_rate_ci95_low", "success_rate_ci95_high"]
    with open(output_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for eps in sorted(results.keys()):
```

```
63                writer.writerow(results[eps])
```

The full script (including CLI argument parsing and wiring to the project's existing `model.py` / `dataset.py` / `attacks.py` modules) is provided as a separate file, `evaluate_full_dataset.py`, alongside this report.

### Summary Statistics – Results Table

The table below is the target output format of the full-dataset run. Values are left as **TBD** pending execution of the script on the full LFW test split; they will be filled in once the run completes.

| $\epsilon$ | N | Mean Cos. Sim. | Mean L2 Dist. | Success Rate | 95% CI |
|---|---|---|---|---|---|
| 0.01 | TBD | TBD | TBD | TBD | TBD |
| 0.03 | TBD | TBD | TBD | TBD | TBD |
| 0.05 | TBD | TBD | TBD | TBD | TBD |
| 0.10 | TBD | TBD | TBD | TBD | TBD |

### Planned Visualizations (Post-Run)

- Line plot of Attack Success Rate vs. epsilon.
- Histogram/KDE of cosine similarity scores, original vs. adversarial, per epsilon.
- Box plot of L2 distance distributions across epsilon values.

## 9. Google Sheets One-Line Progress

### Progress Summary

> Completed literature review on adversarial attacks in face recognition, explored multiple datasets and GitHub repositories, selected the LFW dataset, implemented a ResNet50-based feature extractor and FGSM adversarial attack framework, generated adversarial face images, built a cosine-similarity-based attack evaluation pipeline, and implemented a full-dataset evaluation script with summary-statistics reporting (execution and results pending).

## 10. Next Objectives

- Execute the full-dataset evaluation script on the LFW test split and populate the results table with actual figures.
- Generate the planned visualizations from the CSV output.

- Implement stronger attacks such as PGD and DeepFool.
- Evaluate model robustness under the additional attack settings, using the same evaluation pipeline for a fair comparison.
- Analyze quantitative and qualitative results across all attack methods.
- Prepare the final project documentation and presentation.